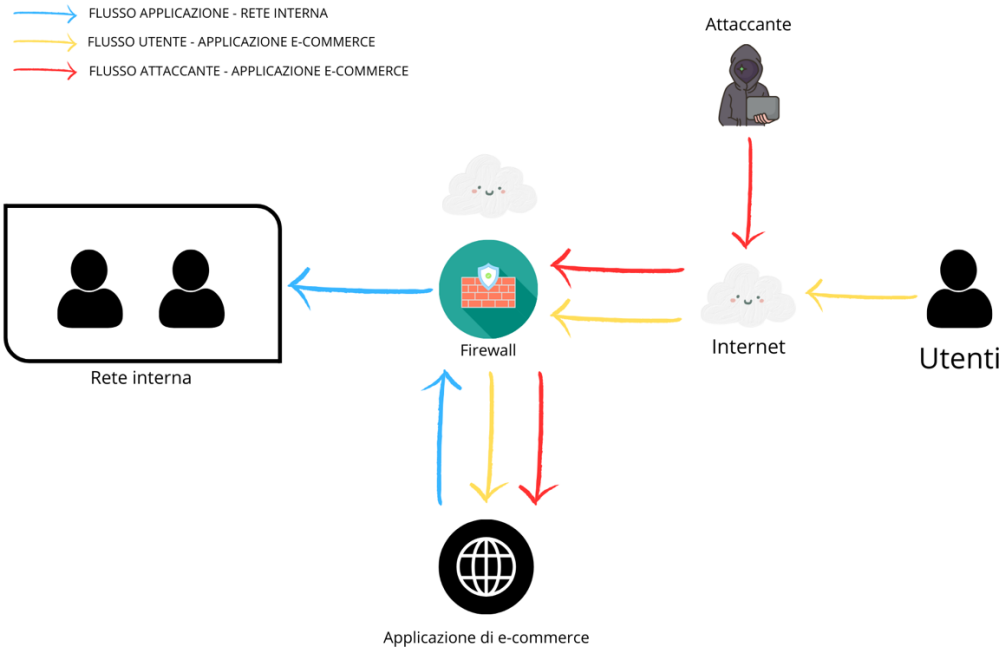
Raul Pastor

PROGETTO FINALE M4



Data la precedente architettura di rete, rispondere ai seguenti quesiti:

1. Azioni preventive: quali azioni preventive si potrebbero implementare per difendere l'applicazione Web da attacchi di tipo SQLi oppure XSS da parte di un utente malintenzionato? Modificate la figura in modo da evidenziare le implementazioni.

2. Impatti sul business: l'applicazione Web subisce un attacco di tipo DDoS dall'esterno che rende l'applicazione non raggiungibile per 10 minuti. Calcolare l'impatto sul business dovuto alla non raggiungibilità del servizio, considerando che in media ogni minuto gli utenti spendono 1.500 € sulla piattaforma di e-commerce. Fare eventuali valutazioni di azioni preventive che si possono applicare in questa problematica.

3. Response: l'applicazione Web viene infettata da un malware. La vostra priorità è che il malware non si propaghi sulla vostre rete, mentre non siete interessati a rimuovere l'accesso da parte dell'attaccante alla macchina infettata. Modificate la figura in slide 2 con la soluzione proposta.

4. Soluzione completa: unire i disegni dell'azione preventiva e della response (unire soluzione 1 e 3) .

5. Modifica «più aggressiva» dell'infrastruttura (se necessario/facoltativo magari integrando la soluzione al punto 2).

1) Azioni preventive:

SQL Injection (SQLi)

Query parametrizzate e Prepared Statements (Misura primaria)

Obiettivo
Impedire che l'input dell'utente venga interpretato come parte della sintassi SQL.

Descrizione tecnica
Tutte le interrogazioni al database devono essere implementate mediante query parametrizzate o prepared statements, evitando la concatenazione diretta di input utente all'interno delle stringhe SQL. La struttura della query deve essere definita staticamente, mentre i valori forniti dall'utente devono essere passati come parametri separati, così da essere trattati esclusivamente come dati e non come codice eseguibile. Anche in presenza di ORM è necessario evitare query raw o costruzioni dinamiche non controllate.

Motivazione tecnica
Nei prepared statements il database distingue chiaramente:

- Struttura della query (codice SQL)
- Valori dei parametri (dati)

Questo impedisce all'input malevolo di alterare la struttura della query.

Validazione Input (Approccio Allow-List)

Obiettivo
Ridurre la superficie di attacco accettando solo input conformi a regole attese.

Descrizione tecnica
Ogni dato ricevuto dall'esterno deve essere validato lato server verificando che rispetti formato, tipo e vincoli previsti dal modello applicativo. L'applicazione deve accettare esclusivamente valori conformi a regole predefinite, come numeri entro un certo intervallo o parametri di ordinamento appartenenti a un insieme chiuso di opzioni. Questo riduce la possibilità che input malevoli raggiungano il database.

Motivazione tecnica
La validazione preventiva intercetta payload anomali prima che raggiungano il livello di accesso al database.

Principio del Least Privilege sul Database

Obiettivo
Limitare l'impatto nel caso in cui una vulnerabilità venga sfruttata.

Descrizione Tecnica

L'account utilizzato dall'applicazione per accedere al database deve disporre solo dei privilegi strettamente necessari alle operazioni previste. Devono essere esclusi permessi

amministrativi o strutturali non indispensabili. In questo modo, anche in caso di sfruttamento di una vulnerabilità SQL Injection, l'impatto sull'integrità del sistema risulta limitato.

Motivazione tecnica
Anche in caso di SQLi riuscita, l'attaccante non deve poter:

- Eliminare tabelle
- Accedere a database non previsti
- Fare privilege escalation

Web Application Firewall

Obiettivo
Intercettare pattern di attacco noti prima che raggiungano l'applicazione.

Implementazione

- WebApplicationFirewall in DMZ davanti al server applicativo
- Attivazione regole anti-SQLi

XSS

Output Encoding Contestuale (Misura primaria)

Obiettivo
Impedire l'esecuzione di script iniettati nel browser dell'utente.

Descrizione tecnica
Tutti i dati provenienti dall'utente devono essere sottoposti a encoding in fase di output, in base al contesto in cui vengono inseriti nella pagina HTML. I caratteri con significato sintattico devono essere convertiti in entità sicure, impedendo che vengano interpretati come codice eseguibile dal browser.

Motivazione tecnica
Il dato viene trattato come testo e non come codice eseguibile.

Content Security Policy (CSP)

Obiettivo
Ridurre l'impatto di eventuali XSS non intercettate.

Descrizione tecnica

L'applicazione deve inviare header HTTP di Content Security Policy che limitino le sorgenti da cui il browser può caricare ed eseguire script. La restrizione degli script inline e delle sorgenti non autorizzate riduce l'impatto di eventuali vulnerabilità XSS.

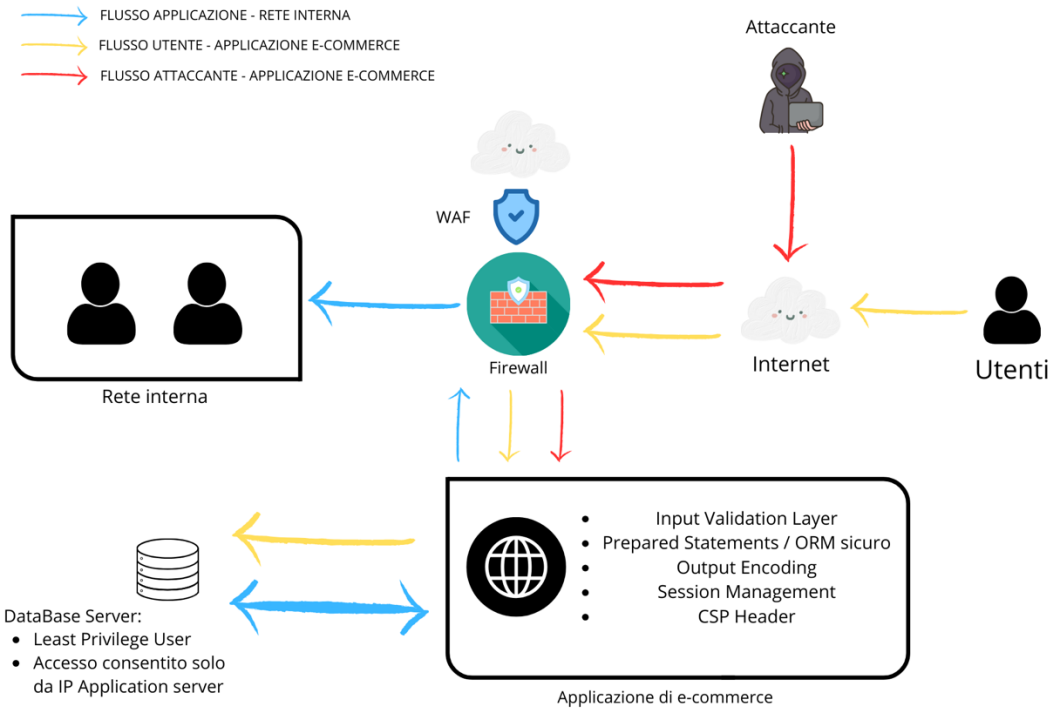
Motivazione tecnica
Anche se un payload viene iniettato, il browser ne blocca l'esecuzione.

Cookie Sicuri e Gestione Sessione

Obiettivo
Limitare l'account takeover in caso di XSS.

Descrizione tecnica
I cookie di sessione devono essere configurati con gli attributi HttpOnly, Secure e SameSite, così da limitarne l'accesso tramite script e la trasmissione su canali non cifrati. Devono inoltre essere previsti meccanismi di timeout e rotazione della sessione per ridurre il rischio di utilizzo improprio in caso di attacco.

Nota tecnica
L'XSS può comunque eseguire azioni per conto dell'utente autenticato; questa misura riduce il furto diretto del cookie.



2) Impatti sul business (attacco DDoS):

Calcolo della perdita economica diretta

- Ricavo medio al minuto: **1.500 €**
- Durata indisponibilità: **10 minuti**

Calcolo:

1.500 € x 10(minuti) = 15.000 €

Perdita economica diretta stimata: 15.000 €

Questa cifra rappresenta il mancato fatturato generato durante il periodo di indisponibilità del servizio.

Valutazione delle azioni preventive contro DDoS

- Servizi Anti-DDoS e CDN

L'adozione di un servizio di protezione DDoS basato su rete distribuita di assorbire e filtrare il traffico malevolo prima che raggiunga l'infrastruttura applicativa. Questo riduce il rischio di saturazione delle risorse.

- Architettura scalabile (Auto Scaling)

Un'architettura basata su bilanciatori di carico e scalabilità orizzontale permette di gestire picchi di traffico improvvisi. In ambienti cloud, l'auto scaling consente l'attivazione dinamica di nuove istanze in caso di sovraccarico.

- Load Balancer e ridondanza

L'utilizzo di load balancer distribuisce il traffico su più server applicativi, evitando single point of failure. La ridondanza geografica aumenta ulteriormente la resilienza.

- Rate Limiting e Traffic Filtering

L'implementazione di:

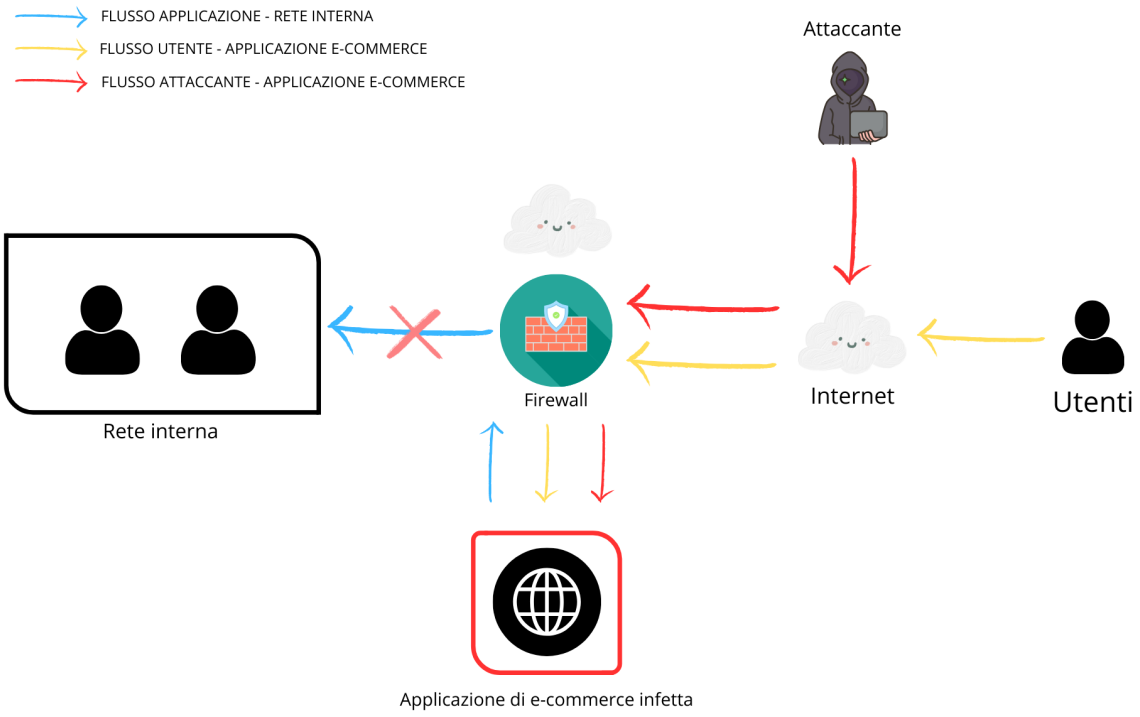
- Limitazioni sul numero di richieste per IP
- CAPTCHA su endpoint critici
- Filtri su traffico anomalo

riduce la possibilità che richieste automatizzate saturino le risorse.

3) Response (Malware Infection):

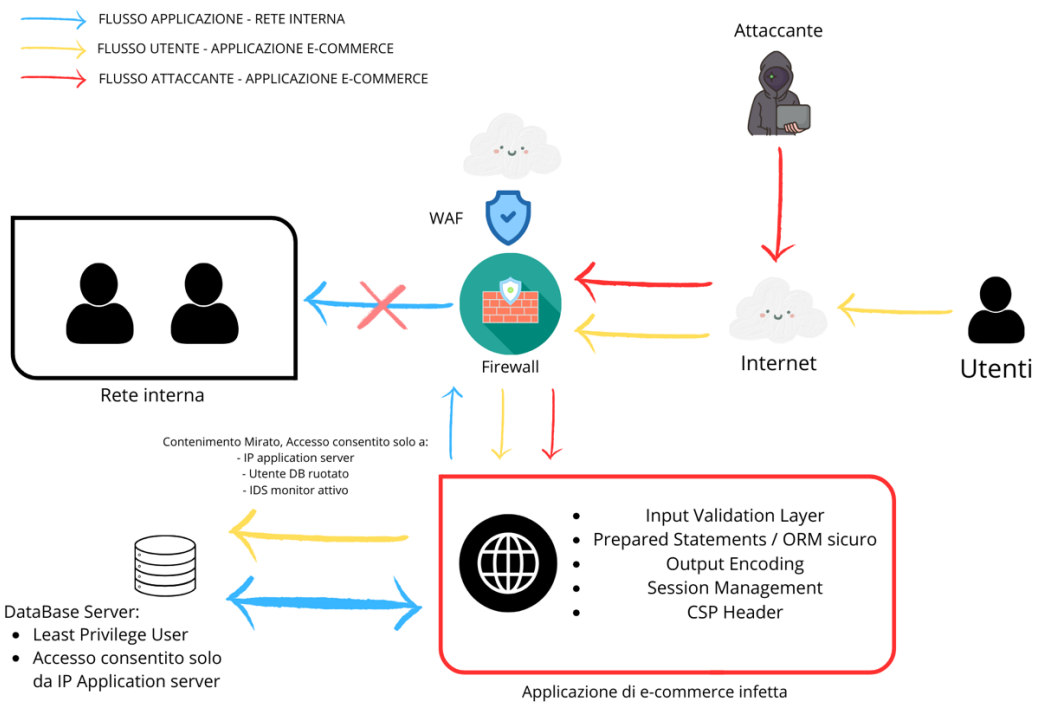
Nella fase immediata: l'obiettivo è impedire la propagazione del malware verso la rete interna. Si procede quindi al blocco del traffico dalla DMZ alla LAN tramite modifica urgente delle regole firewall, adottando un approccio "deny by default" e consentendo solo eventuali flussi strettamente indispensabili. Vengono inoltre revocate le credenziali utilizzate dall'application server per accedere a risorse interne, così da evitare movimenti laterali. Se possibile, il server compromesso viene isolato in una VLAN o rete di quarantena, mantenendolo confinato all'interno della DMZ e impedendo comunicazioni verso altri sistemi interni.

Nel medio e lungo periodo: si effettua un'analisi tecnica dell'incidente per verificare eventuali compromissioni secondarie e comprendere il vettore di attacco. Vengono riviste in modo permanente le policy di segmentazione tra DMZ e rete interna, applicando il principio del minimo privilegio e limitando i flussi solo a quelli strettamente necessari. Si rafforza inoltre il monitoraggio del traffico tra i segmenti di rete e si introducono controlli strutturali, come micro-segmentazione o modelli zero trust, per ridurre il rischio di propagazione in caso di future compromissioni.

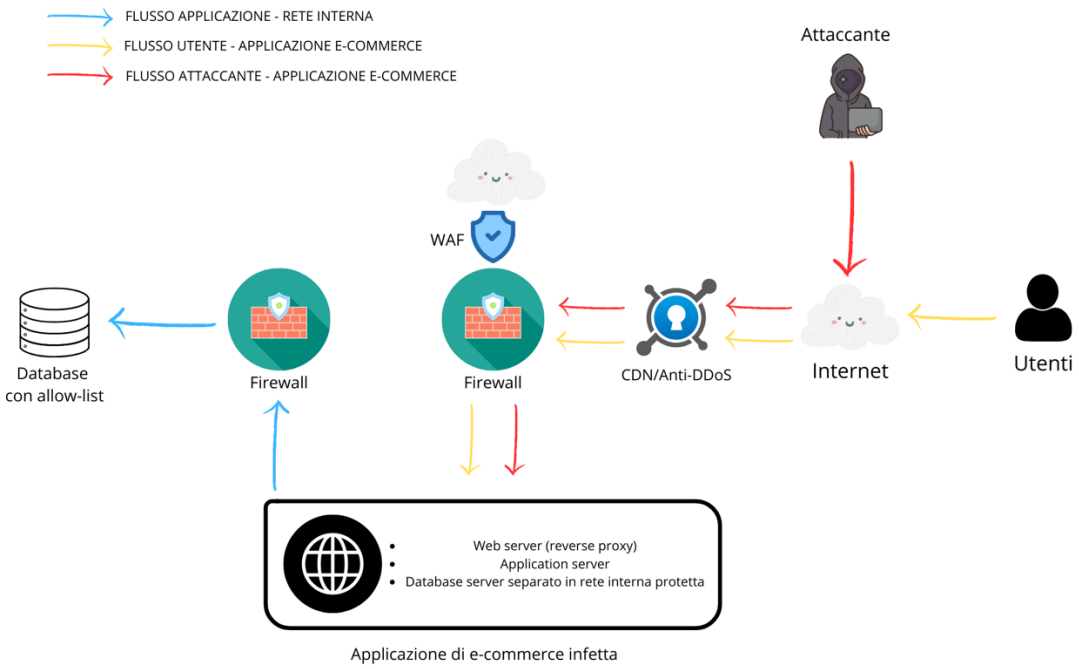


Raul Pastor

4) Soluzione Completa:



5) Modifica aggressiva dell’infrastruttura:

L’architettura è organizzata in più livelli per aumentare sicurezza e continuità del servizio. Il traffico proveniente da Internet passa prima attraverso un sistema di protezione Anti-DDoS che filtra eventuali attacchi e gestisce grandi quantità di richieste.

All’interno della DMZ i server web inoltrano le richieste ai server applicativi, separando così la parte pubblica dalla logica interna del sistema. Il database non è esposto direttamente, ma si trova in una rete separata e protetta da un firewall che consente l’accesso solo ai server autorizzati. In questo modo, anche in caso di compromissione di un componente, l’infrastruttura limita la propagazione dell’attacco e protegge la rete interna.